

11_ai_b

Evaluation

Q26. How do you evaluate an autonomous diagnosis engine when there's no labeled "right answer" lying around in production?

Ideal answer. You manufacture ground truth. The general technique is to take a frozen real dataset, inject anomalies whose root cause you authored, and score whether the system rediscovers exactly what you planted — turning an unmeasurable open-ended task into a closed-form accuracy problem. I built an offline labeled benchmark: a frozen copy of the GA4 dataset into which I perturb specific segments, with a YAML label recording the true cause (which segment, mix vs rate, dollar impact), then run the full pipeline and grade its top-ranked finding against that label. This is strictly better than eyeballing a few outputs because it's repeatable, regression-gateable in CI, and reports a number you can defend. The alternative — LLM-as-judge over unlabeled prod runs — measures plausibility, not correctness, and an engine that confidently invents plausible-but-wrong causes is the exact failure I'm trying to kill. **Why Helios**

demonstrates it. The `helios.eval harness (injector.py -> runner.py -> scorers/)` grades against authored labels in `scenarios/*.yaml` over a frozen `helios_eval` dataset, and the gate lives in CI. **Follow-ups.** Why is a frozen dataset copy essential — what breaks if you inject into live data? / How do you keep labels honest so you aren't grading the system against your own confirmation bias? / What's the cost of maintaining a labeled benchmark as the schema evolves?

Q27. Walk me through synthetic anomaly injection. How do you inject a *mix-shift* differently from a *rate-change*?

Ideal answer. The two perturbation modes map directly onto the decomposition identity $\Delta R = \sum \Delta w_i \cdot r_i(t_0) + \sum w_i(t_0) \cdot \Delta r_i + \sum \Delta w_i \cdot \Delta r_i$, so I inject each by moving exactly one term. A **rate perturbation** holds segment volume shares `w_i` constant and depresses one segment's conversion `r_i` (e.g. drop mobile/Chrome `checkout_to_purchase_rate` by 20% by deleting a fraction of that segment's downstream `reached_*` flags) — true behavior change. A

volume/mix perturbation holds every segment's `r_i` constant and reweights traffic `w_i` (e.g. shift session share from high-converting Direct to low-converting Display by duplicating/dropping whole sessions) — composition artifact with no behavior change. The discipline is that injection touches *one* lever so the label is unambiguous: a correct system must attribute a rate injection to `rate_effect` and a mix injection to `mix_effect`, and the naive "biggest absolute segment delta" baseline conflates them. The label stores the segment, the mode, the magnitude, and the

derived dollar-at-risk so I can score cause, classification, *and* MAPE. **Why Helios demonstrates it.** `eval/injector.py` implements exactly two primitives — rate-perturbation (mutate `r_i` via `reached_*` flags) and volume-perturbation (mutate `w_i` via session resampling) — so each scenario's label is a clean mix-or-rate ground truth. **Follow-ups.** How do you prevent an injection from also accidentally moving a parent metric and contaminating a "single-cause" label? / Why inject by editing `reached_*` flags rather than the final rate column? / How do you inject a seasonality *decoy* that should NOT be flagged?

Q28. Describe the 50-scenario suite. Why those buckets, and what failure does each one police?

Ideal answer. Coverage matters more than count — a benchmark is only as good as the failure modes it can catch, so I designed seven buckets, not 50 random anomalies. They are: single-segment rate change (the base case), multi-segment rate change (additivity under several simultaneous drops), single-segment mix shift and multi-segment mix shift (Simpson's-paradox traps), seasonality decoys (a real-looking dip that is calendar-explained and must NOT be alarmed), no-anomaly controls (the system must stay silent — guards false-positive rate), and data-quality artifacts (a tracking gap or duplicate that looks like a drop but is an ingestion bug). The mix buckets specifically police the engine's core thesis: a baseline that chases the largest delta will misattribute composition artifacts to behavior. The controls and decoys police the opposite failure — an always-on agent that cries wolf gets muted, so silence-when-correct is a graded outcome, not an afterthought. **Why Helios demonstrates it.** The `scenarios/` set spans all 7 buckets; the no-anomaly and seasonality-decoy buckets are what let me report a real precision number and prove the Critic earns its keep. **Follow-ups.** Why include data-quality artifacts at all — isn't that the warehouse's job? / How do you weight buckets when computing a headline score? / What's the minimum scenarios-per-bucket for the accuracy number to be statistically meaningful?

Q29. What exactly do you measure per run? Give me the metric definitions, not just names.

Ideal answer. Five graded quantities, each tied to a trust pillar. (1) **Root-cause segment accuracy** = fraction of scenarios where the top-1 promoted finding names the injected segment *and* the correct mix/rate classification (the headline). (2) **Decomposition MAPE** = mean absolute percentage error of `stats-mcp`'s reported mix/rate/interaction magnitudes against the injected dollar/rate deltas — graded $\leq 10\%$. (3) **Detection F1** over the anomaly-vs-control split, so missed anomalies and false alarms both cost. (4) **Hallucination rate** = count of columns/metrics in emitted SQL absent from the governed registry — a hard-zero gate. (5) **Faithfulness** = fraction of numeric claims in the Decision Brief that are verbatim tool outputs traceable to an `audit_log` row, not paraphrases. Accuracy is the product number; MAPE validates the math pillar; F1 validates detection; hallucination and faithfulness validate the grounding pillar. **Why Helios**

demonstrates it. Each metric is a scorer under `eval/scorers/` (`accuracy.py` , `decomposition.py` , `detection.py` , `hallucination.py` , `faithfulness.py`); the headline targets are $\geq 85\%$ accuracy, $\leq 10\%$ MAPE, 0 hallucinations. **Follow-ups.** Why top-1 and not top-3 recall for the headline? / How do you score a finding that's right on segment but wrong on dollar magnitude? / How is faithfulness scored mechanically without another LLM in the loop?

Q30. Define the naive baseline precisely. Why 85% vs 45%, and why is that gap the whole point?

Ideal answer. The baseline must be the thing a competent analyst or a junior tool would actually do, or the comparison is a strawman. Mine is "**largest absolute segment delta**": cross-tab the metric by each dimension, rank segments by `|\Delta_{segment}|` , and report the biggest mover as the cause — no decomposition, no significance, no mix/rate split. It scores around 45% because it gets single-segment rate changes roughly right but is *systematically* fooled by mix shifts (it flags the segment whose volume moved, not the one whose behavior moved) and by small-but-significant changes it under-ranks. Helios hits $\geq 85\%$ because `decompose_change` separates the levers and Diagnose drills rate-effects first. The gap is the entire product thesis quantified: the hard part isn't surfacing a number, it's attributing it correctly, and 40 points of accuracy is the dollar value of doing the decomposition properly. **Why Helios demonstrates it.** The baseline runs in the same harness over the same scenarios, so `85 vs 45` is an apples-to-apples number, and the mix buckets are precisely where the gap opens. **Follow-ups.** Could you make a stronger baseline (e.g. delta normalized by segment volume) and does the gap survive? / On which bucket does the baseline come closest to Helios? / Why is "largest delta" the right strawman for *this* product rather than a random baseline?

Q31. How does eval run in CI, and what exactly does it gate?

Ideal answer. A benchmark that isn't a merge gate is a vanity metric, so I wired it into GitHub Actions with two tiers. Every push runs a **smoke subset** (a fast representative slice across buckets) for quick signal; every PR to `main` runs the **full suite**. The gate (`eval/gates.yaml`) blocks the merge if top-1 accuracy regresses more than ~2 points from the baseline-on-main, if hallucination is anything but zero, if decomposition MAPE exceeds 10%, or if a run breaks the 5 GiB byte budget or the <5-min latency SLA. Crucially the *same* `hallucination.py` scorer that gates CI also runs over the production `audit_log` , so the offline invariant and the online observation are enforced by one piece of code. This makes correctness a property the repo protects mechanically, not a claim that quietly rots. **Why Helios demonstrates it.** `.github/workflows/ci.yaml` invokes `eval.runner --smoke` on push and `--full` on PR-to-main; `eval/gates.yaml` encodes the accuracy/hallucination/MAPE/budget thresholds as hard gates. **Follow-ups.** Why a 2-point regression tolerance and not zero? / How do you keep full-

suite eval cost inside a CI budget given BigQuery scans? / How do you stop the benchmark from being silently overfit by repeated tuning against it?

Q32. What stops you from overfitting the agents to your own 50 scenarios?

Ideal answer. A benchmark you tune against eventually measures memorization, so I treat it like a held-out set with hygiene rules. I keep the injection *primitives* generic (perturb any (segment, mode, magnitude)) rather than hand-crafting bespoke cases the agents could pattern-match, and I parameterize scenarios so I can resample fresh segment/magnitude combinations the prompts have never seen. I separate a smoke subset used during development from the full suite that gates merges, and I watch per-bucket scores, not just the headline, so improving rate-cases by breaking mix-cases shows up immediately. The honest limitation I'd state in an interview: 50 scenarios over one observational dataset is a starting harness, not a guarantee of generalization, and the right next step is randomized scenario generation plus a second dataset. **Why Helios demonstrates it.** The *injector* is primitive-based and parameterized rather than a fixture of frozen answers, and the smoke/full split (--smoke vs --full) keeps a portion out of the daily tuning loop. **Follow-ups.** How would you build a true train/test split for scenarios? / What signal tells you you've started overfitting? / Would you trust the 85% on a second e-commerce dataset, and why or why not?

Q33. Why is an offline benchmark a stronger artifact than a slick live demo?

Ideal answer. A demo proves the system *can* be right once; a benchmark proves how often it *is* right, and the second is what earns trust for an unattended agent. Live demos are cherry-picked, non-reproducible, and silent about failure modes — they answer "does it look impressive" not "what's the error rate on mix-shift traps." The benchmark gives me a defensible distribution: 85% top-1, zero hallucinations, MAPE under 10%, against a stated 45% baseline, reproducible on every commit. For autonomous tooling specifically this matters more than for a chatbot, because no human is in the loop to catch a wrong answer at runtime — the only place correctness can be established is offline, in advance. In an interview the benchmark also lets me discuss failure honestly, which reads as senior judgment rather than salesmanship. **Why Helios demonstrates it.** The headline is reported as a graded distribution from *eval.runner* , not a screenshot — the trust story is "measured 85% vs 45%," which is the third defensibility pillar (adversarial verification + offline eval). **Follow-ups.** When is a live demo the better artifact? / How do you communicate eval results to a non-technical stakeholder? / What would change your headline metric if a customer's funnel looked nothing like the Merch Store?

Prompt Engineering

Q34. How are the system prompts structured across the seven agents — one mega-prompt or per-agent?

Ideal answer. Per-agent, narrow, and role-scoped — a single mega-prompt invites the model to do everything badly and makes failures unattributable. Each agent gets a system prompt that states only its job, its allowed tools, its output contract, and the rules it must obey, so the prompt surface for any one node is small and testable. The Monitor's prompt knows only "call `detect_anomaly` , emit a typed Finding or 'clean'"; the Diagnose prompt knows the best-first search policy and the rate-before-mix ordering; the Critic's prompt is purely adversarial — "try to refute this finding via these four attack templates." The shared spine across all prompts is the grounding contract (G1-G5) and the typed "Finding" JSON envelope every agent reads and writes. Smaller role-scoped prompts also let me pick model tier per agent (Opus for Diagnose/Critic/Orchestrator reasoning, Sonnet for the mechanical nodes), which is both cheaper and more reliable. **Why Helios demonstrates it.** Seven distinct agents (`helios/agents/`) each carry a focused system prompt plus a per-agent MCP tool allow-list (Bible §18.9) — e.g. the Narrator's prompt and allow-list both forbid `run_query` . **Follow-ups.** How do you keep seven prompts consistent without copy-paste drift? / Why give Diagnose and Critic Opus but Narrator Sonnet? / How do you version and test a prompt change without a full eval run?

Q35. State the grounding rules G1-G5 and explain which are behavioral and which are structural.

Ideal answer. **G1** never emit raw SQL — get data only via `semantic-mcp.build_query / get_metric` . **G2** never compute a statistic in prose — anomaly, decomposition, significance, forecast, power come only from `stats-mcp / experiment-mcp` , and numbers in the brief are tool outputs verbatim. **G3** `warehouse-mcp.dry_run` before every `run_query` ; over-budget means narrow the query, not retry blindly. **G4** reconcile aggregates via `warehouse-mcp.reconcile` ; >0.5% drift fails the finding. **G5** use only canonical metric/dimension names — an unknown name is a hard error, never a fallback to free SQL. The distinction that matters: stating these in a prompt is *behavioral* (the model is asked to comply) but Helios also enforces them *structurally* — the agent literally has no SQL tool, only `semantic-mcp` ; the per-agent allow-list makes `run_query` uncallable from a writer agent; the byte gate and reconcile run in `warehouse-mcp` regardless of what the model "intends." The prompt makes compliance the path of least resistance; the architecture makes violation impossible. **Why Helios demonstrates it.** G1-G5 are documented as the five non-negotiables in `CLAUDE.md` , and each has a structural counterpart: no raw-SQL tool exists, allow-lists gate tools, `dry_run / reconcile` are mandatory in `warehouse-mcp` . **Follow-ups.** Which of G1-G5 would still hold if the prompt were blank, and which wouldn't? / What's the failure mode if G3's `dry_run` is advisory rather than enforced? / How does G4's 0.5% threshold interact with floating-point rounding in the warehouse?

Q36. Why do you claim "structural beats prompt-only"? Convince me a prompt isn't enough.

Ideal answer. A prompt is a *request* to a probabilistic system; under distribution shift, long context, or an adversarial input it will eventually be violated, and for an unattended agent "eventually" means "in production, unobserved." The senior move is to make the desired property a structural invariant so violation isn't a low-probability event, it's impossible. Concretely: I can ask the model in a prompt to "only use governed metrics" (prompt-only) — or I can give the agent no SQL-execution tool at all and route every data request through `semantic-mcp`, so the model *cannot* express a hallucinated column even if it wanted to (structural). Prompt-only gets you maybe 99% compliance; structural gets you 100% and, more importantly, gets you *provability* — I can scan `audit_log.sql_text` and assert every column came from the registry. The tradeoff is real: structural governance is more upfront engineering than a paragraph of instructions, and it constrains what the model can do, which is exactly why it's safe. **Why Helios demonstrates it.** Helios's hallucination rate is a hard-zero CI gate, achievable only because `semantic-mcp` is the *sole* SQL path and the registry is the single source of truth — the model never holds a SQL pen. **Follow-ups.** Give an example where prompt-only grounding silently fails at 99% compliance. / Does structural governance ever block a legitimate analysis the model should do? / How do you add a new metric without loosening the structural gate?

Q37. Walk me through writing the Critic's prompt. How do you prompt a model to attack rather than agree?

Ideal answer. LLMs are sycophantic by default — they want to ratify the finding handed to them — so the Critic's prompt inverts the objective: its job is to *refute*, and a finding survives only by surviving attack. I give it a fixed set of attack templates rather than open-ended "is this right?", because named adversarial moves are testable and reproducible: mix-confound (is this composition not behavior?), insufficient-sample (does `significance_test` actually clear?), seasonality (is this calendar-explained?), and data-quality (is this a tracking artifact?). The prompt forces a structured verdict — PASS, DOWNGRADE, or DROP — with a cited reason, and routes every check through tools (re-run `decompose_change`, consult the `seasonality_calendar`) so the refutation is grounded, not rhetorical. Framing it as "earn the finding's survival" rather than "grade the finding" is what flips the model out of agreement mode. **Why Helios demonstrates it.** The Critic (Opus) runs four named refutation templates and emits a PASS/DOWNGRADE/DROP verdict per the Finding envelope; it consults memory's `seasonality_calendar` / `launch_calendar` and re-runs `stats-mcp` rather than judging in prose. **Follow-ups.** How do you stop the Critic from being so aggressive it drops every real finding? / How do you eval the Critic in isolation? / Why named attack templates instead of a free-form adversarial prompt?

Q38. What does refusal of free SQL look like in practice — show me the path when the model wants a column that doesn't exist.

Ideal answer. The model never gets to "want a column" in SQL terms because the only data verb it can call is `semantic-mcp.build_query(metric, dims, filters, window)` over canonical names. If it composes a call referencing a name not in the registry, `semantic-mcp` returns a hard error listing the valid names — there is no escape hatch that drops to hand-written SQL (G5). For example:

```
build_query(metric="add_to_cart_conversion", dims=["device"])
→ ERROR: unknown metric "add_to_cart_conversion" (did you mean "view_to_cart_rate?");
unknown dimension "device" (did you mean "device_category?"). No SQL emitted.
```

The agent's correct response is to re-compose with valid names, not to route around the gate, because no around exists — there's no `run_sql` tool in its allow-list. This is the structural enforcement of G1/G5: a hallucinated column is caught at compose time as a typed error, never reaching the warehouse, which is why the eval can claim a hard zero on hallucinations. **Why Helios demonstrates it.** `semantic-mcp.build_query` validates every metric/dimension against `semantic_models.yml` (28 metrics + 16 dimensions, referential-integrity compiled) and errors on unknowns; no agent has a raw-SQL tool. **Follow-ups.** Should the error suggest near-matches, and does that risk the model rationalizing a wrong metric? / What if the analysis genuinely needs a metric not yet in the registry? / How does this differ from a text-to-SQL system's "validate after generate" approach?

Q39. How do you prompt for the typed "Finding" envelope, and why a typed handoff at all?

Ideal answer. Free-text handoffs between agents lose structure and let errors compound silently, so every agent reads and writes one typed JSON "Finding" — segment, metric, mix/rate classification, magnitudes, significance, dollar impact, Critic verdict, and provenance. The prompt specifies the schema and instructs the agent to emit *only* valid JSON conforming to it, and I validate the output against the schema before the FSM advances — a malformed envelope is a node failure, not a downstream surprise. This makes the pipeline composable (Diagnose's output is exactly Critic's input), debuggable (every field is inspectable in `audit_log`), and lets the deterministic FSM, not prose, control flow. The envelope is also what decouples the agents from the data: they pass typed findings, never rows, which is why `stats-mcp` and the workers are stateless and replicable. **Why Helios demonstrates it.** The typed "Finding" envelope is the contract handed between the seven agents on the FSM; validating it at each node is what lets the Orchestrator drive flow deterministically rather than parsing model prose. **Follow-ups.** What do you do when the model emits prose around the JSON? / How do you evolve the Finding schema without breaking older diagnoses in memory? / Why JSON over a tool-call/function-calling structured output?

Memory Systems

Q40. Describe Helios's memory. What's persisted, and in what stores?

Ideal answer. Memory turns a one-shot analyst into one that learns across runs, so I persist four things in `helios_memory` (BigQuery) plus a vector store. (1) **Diagnosis history** — every shipped finding with its embedding, so a new finding can be matched against priors semantically, not just by exact segment match. (2) A **suppression list** — acknowledged or auto-suppressed findings with a reason and TTL, so the engine stops re-alarms on known issues. (3) **Seasonality and launch calendars** — known periodic effects and product/marketing events the Critic uses to refute calendar-explained dips. (4) An **action-tracking** table — the prescribed experiment, its status, and the measured readback, closing the did-the-fix-work loop. BigQuery holds the structured rows and durable state of record; the vector store holds embeddings for semantic recall. Crucially, all cross-run learning lives in these stores, never in an agent's heap, which is what keeps the agents stateless and the system restartable. **Why Helios demonstrates it.**

`helios_memory` + vector store hold diagnosis history (embeddings), suppression list, seasonality/launch calendars, and action-tracking; `report-mcp` exposes

`save_diagnosis` / `recall_prior` as the memory interface. **Follow-ups.** Why split structured rows (BigQuery) from embeddings (vector store)? / What's the schema of the diagnosis-history row? / How does keeping state out of the agent heap help horizontal scale?

Q41. How does `recall_prior` work, and how does it change a diagnosis?

Ideal answer. `recall_prior` is semantic retrieval over diagnosis history: given the current finding (segment, metric, movement), it embeds the query and returns the nearest prior diagnoses plus their outcomes and any suppression/calendar matches. This changes the diagnosis three ways — it lets Diagnose say "we've seen this mobile-Safari checkout drop before, here's what we found and whether the fix worked," it lets the Critic check whether this exact issue is suppressed or calendar-explained before promoting it, and it lets the Narrator add continuity ("recurring, third time this quarter") instead of treating every run as amnesiac. The retrieval is semantic so it matches *related* priors (a Safari issue surfacing the prior Chrome-mobile finding), not just byte-identical ones. The guardrail: recall *informs* but never *overrides* the live decomposition — priors are evidence the current tools must still confirm, so a stale prior can't manufacture a finding the data doesn't support. **Why Helios demonstrates it.** `report-mcp.recall_prior` performs embedding-based retrieval over the diagnosis history; Diagnose and the Critic both consult it before promoting a leaf. **Follow-ups.** How do you stop a confident prior from biasing the model toward a wrong-but-familiar cause? / What's your embedding model and similarity threshold? / How do you handle a prior whose schema/metrics have since changed?

Q42. Explain the suppression list and its TTL. Why is "don't re-alarm" a memory problem?

Ideal answer. An always-on engine that re-reports the same known issue every run gets muted by its operators — so suppression is the difference between a tool people keep running and one they turn off. When a stakeholder acknowledges a finding (or the Critic auto-suppresses a known artifact), I write a suppression row with the segment/metric signature, a human reason, and a TTL (default 30 days). Diagnose and Critic consult it before promoting a leaf; a match is auto-DROPPed with reason "suppressed." The TTL is the subtle part: suppression must *expire* so a genuinely re-emerging problem eventually re-surfaces rather than being silenced forever — a permanent mute is how you miss the second, real occurrence. This is a memory problem because it's cross-run state about operator intent, which can't live in a stateless agent or be re-derived from the data alone. **Why Helios demonstrates it.** The TTL'd suppression list in `helios_memory` is consulted by Diagnose and the Critic; a match yields an auto-DROP verdict with reason "suppressed." **Follow-ups.** Why 30 days and not indefinite or 7 days? / How do you distinguish "suppress this instance" from "suppress this pattern forever"? / What happens if a suppressed issue's dollar impact suddenly triples within the TTL?

Q43. How does the action-tracking loop close, and why does it make Helios more than a reporter?

Ideal answer. A diagnosis without a feedback loop is a reporter; closing the loop is what makes it an analyst that's accountable for its prescriptions. When Prescribe ships an experiment card, I write an action-tracking row (the recommended action, the targeted segment/metric, the expected effect, status). Because the dataset is observational, Helios can't run a live A/B, so it closes the loop with a *quasi-experimental readback* — pre/post or difference-in-differences on the segment after the action's stated date — and writes the measured outcome back to the row. That outcome then feeds future `recall_prior` calls, so the next time a similar issue appears the engine knows whether its prior fix actually worked. This is the did-the-fix-work loop, and it's what lets Helios improve its prescriptions over time instead of repeating advice that didn't land. **Why Helios demonstrates it.** Action-tracking in `helios_memory` plus the honest constraint that the data is observational (Nov 2020-Jan 2021) — so Helios designs/sizes experiments and runs pre/post or diff-in-diff readbacks rather than live A/Bs. **Follow-ups.** Why is diff-in-diff a defensible readback on observational data, and where does it break? / How do you attribute a metric move to your action vs confounders? / What do you do when the readback says the fix didn't work?

Q44. Memory grows unbounded. How do you handle decay, staleness, and relevance over time?

Ideal answer. Unbounded memory degrades both cost and quality — stale priors mislead recall and the byte budget balloons — so memory needs decay just like a cache. Suppression entries decay via their TTL. Diagnosis history I'd weight by recency and by whether the action-tracking readback confirmed the diagnosis, so a confirmed recent finding outranks an old unconfirmed

one in `recall_prior`; outright pruning or archiving very old, low-relevance diagnoses keeps the vector index lean. Seasonality entries are inherently periodic so they persist, but launch-calendar entries age out once their effect window passes. The principle is that recall relevance should reflect *current* truth, and that means actively forgetting: a year-old finding about a since-redesigned checkout is noise, not signal. Honestly, at 3 months of data this is more design-for-scale than a present pain, but it's the right thing to build before memory is large. **Why Helios demonstrates it.** TTL'd suppression plus recency/confirmation-weighted recall over the embedded diagnosis history — decay is a first-class property of the memory schema, not an afterthought. **Follow-ups.** How would you measure whether decay is helping or hurting recall quality? / Should a *contradicted* prior be deleted or kept as a negative example? / How do you bound the vector store's growth in a multi-tenant deployment?

Architecture Decisions

Q45. The biggest decision: grounding over generation. Defend it against "just let a strong model write the SQL."

Ideal answer. Letting a strong model free-text SQL optimizes for *capability demo*; grounding optimizes for *correctness at scale*, and for an unattended engine the second is the only one that matters. A model writing SQL will, at some rate, hallucinate a column, mis-define a rate as an average-of-ratios (re-introducing Simpson's paradox), or silently change a metric's definition between runs — and no one is watching at 6am. Grounding routes every query through `semantic-mcp` over a governed registry, so columns are provably real, definitions are stable, and the same `revenue` means the same thing every run. The tradeoff is honest: I gave up the model's flexibility to express arbitrary novel queries, and I pay upfront cost building the semantic layer. But that's exactly the trade that buys a hard-zero hallucination gate, multi-tenant scale (add a registry, not a fork), and reproducibility. The thesis: the hard part of an AI analyst isn't generating insights, it's making them correct and trusted — and you can't trust SQL you can't prove. **Why Helios demonstrates it.** `semantic-mcp` as the sole SQL path over `semantic_models.yml` is grounding pillar #1; it's what makes "0 hallucinated columns" a CI gate rather than a hope. **Follow-ups.** What's the largest class of legitimate query your grounded approach *can't* serve? / How do you keep the registry from becoming a bottleneck on analyst velocity? / Where would you allow generated SQL, if anywhere?

Q46. "Determinism where it matters" — where do you allow non-determinism, and where do you forbid it?

Ideal answer. The rule is: non-determinism is fine for *search*, forbidden for *truth*. I let the model be non-deterministic in which hypotheses it explores — that's creative search over a combinatorial space and variance there is acceptable because the Critic and significance gates filter it. I forbid non-determinism in three places: the math (all

decomposition/significance/forecast/power run in seeded Python in `stats-mcp`, reproducible byte-for-byte), the SQL (emitted from a fixed registry, so two runs reference the same columns), and the control flow (a deterministic FSM, not the model, decides node order). So the model is a *composer inside a deterministic shell*: its variance is confined to hypothesis selection, and every number in the brief is a verbatim tool output reconstructable from `audit_log`. The failure I'm avoiding is "the AI said so" — a system where you can't tell whether a different answer means new data or just a different sample from the model. **Why Helios demonstrates it.** Deterministic FSM + seeded `stats-mcp` + governed `semantic-mcp` confine non-determinism to which hypotheses Diagnose explores, which the Critic then filters. **Follow-ups.** How do you make `scipy/statsmodels/prophet` runs actually reproducible? / If hypothesis selection varies, can two runs disagree on the top finding — and is that acceptable? / Where would more determinism hurt rather than help?

Q47. Build vs buy: you used Claude Agent SDK and MCP but built your own semantic layer, stats, and eval. Why that line?

Ideal answer. I bought the commodity and built the moat. The agent runtime, tool-calling protocol (MCP), and CI are commodities — well-solved, not differentiating — so I used the Claude Agent SDK and MCP rather than reinventing an orchestration framework. But the things that make Helios *trustworthy* are the differentiators, and those I built: the governed semantic layer (mapping 1:1 to dbt MetricFlow, so I'm not even reinventing metric definitions — I'm aligning with the standard), the deterministic stats in real Python, and the offline labeled eval. I also deliberately *didn't* buy a generic text-to-SQL or BI tool because their "validate after generate" model can't give me a hard-zero hallucination guarantee. The principle: build where correctness and trust live, buy where the problem is already solved, and don't build a framework when a protocol exists. **Why Helios demonstrates it.** Stack = bought MCP + Claude Agent SDK + GitHub Actions; built `semantic_models.yml` (1:1 to MetricFlow), `stats-mcp`, and the `helios.eval` harness — the three trust pillars. **Follow-ups.** Why MCP over a hand-rolled tool interface? / Would you have used a managed semantic layer (LookML, MetricFlow) if one fit? / What would you buy next as the project grows?

Q48. Cost control: how do you keep an autonomous, multi-agent, LLM-plus-BigQuery system from getting expensive?

Ideal answer. Cost in this system has two axes — warehouse bytes and model tokens — and I bound both with hard gates, not vigilance. On bytes: a mandatory `dry_run` precedes every `run_query` (G3), the run must fit a fixed 5 GiB budget, marts are incremental and partitioned/clustered so queries scan partitions not full history, and over-budget means *narrow the query*, not retry. On tokens: I tier models (Opus only for the three reasoning-heavy agents — Orchestrator, Diagnose, Critic — Sonnet for the mechanical nodes), the best-first search is

depth/breadth-capped (`MAX_DEPTH=3` , `MAX_BRANCHING=4`) so it doesn't explore a combinatorial blowup, and the typed-envelope handoffs keep context small rather than passing rows. The byte budget is also a CI gate, so a change that makes runs scan more fails the build. The principle: make cost a *budget the system enforces on itself*, not a bill you discover later. **Why Helios demonstrates it.** The `dry_run` -> budget gate in `warehouse-mcp` , incremental partitioned marts, per-agent model tiering (Opus/Sonnet), and capped best-first search; the 5 GiB budget is gated in CI. **Follow-ups.** What's your single biggest token cost and how would you cut it? / How does the `dry_run` gate behave when a legitimate query is genuinely large? / How would cost scale with tenants, and what's the per-run unit economics?

Q49. What would you change if you rebuilt Helios from scratch today?

Ideal answer. Three things, in priority order. First, the eval: 50 scenarios over one observational dataset is a harness, not a guarantee, so I'd build randomized scenario generation and add a second, structurally different dataset to test generalization — the 85% is honest but single-dataset. Second, identity: the dataset's `user_id` is almost always null so everything is cookie-grain (`user_pseudo_id`), and I'd design the identity layer to gracefully upgrade to true user-level metrics where a stitching key exists, rather than baking cookie-grain assumptions throughout. Third, the experiment loop: because the data is historical I can only do quasi-experimental readbacks (pre/post, diff-in-diff), so on a live deployment I'd wire real A/B integration so Prescribe's cards close the loop with actual experiments, not just designs. None of these are bugs — they're the honest edges of an observational-data portfolio project, and naming them is the point. **Why Helios demonstrates it.** These map to stated limitations — single-dataset eval, cookie-grain identity (null `user_id`), and observational-only readbacks — which Helios surfaces rather than hides. **Follow-ups.** Of those three, which most threatens the 85% claim? / What would you NOT change because it's already right? / How would real A/B integration change the Critic's job?

Q50. Tie it together: how do the three defensibility pillars and the eval/Critic/memory components make this *trustworthy*, not just impressive?

Ideal answer. Trust is the product, and it's the sum of three pillars each backed by a concrete component. Pillar one — governed-SQL grounding: `semantic-mcp` is the only SQL path, so every column is provably real and the hallucination gate is a hard zero. Pillar two — deterministic statistics: `stats-mcp` is the only math path, seeded, so the decomposition that separates mix from rate is reproducible and the brief's numbers are verbatim tool outputs. Pillar three — adversarial verification plus offline eval: the Critic tries to refute every finding via named attacks, and the labeled benchmark grades the whole pipeline at 85% vs a 45% baseline, in CI, on every commit. Memory makes it accountable over time — suppression stops crying wolf, the action-tracking loop checks whether fixes worked. The synthesis: dashboards tell you *what* and ask you

to trust them; Helios tells you *why*, prices it in dollars, prescribes a powered experiment — and then *proves* each step is correct. The hard part was never generating insight; it was making insight you can put your name on. **Why Helios demonstrates it.** The three pillars (`semantic-mcp` grounding, `stats-mcp` determinism, Critic + 85%-vs-45% eval) plus `helios_memory` (suppression + action-tracking) are the entire trust architecture — every impressive output is gated by a verification. **Follow-ups.** If you could keep only one pillar, which earns the most trust per unit of effort? / How would you explain the trust story to a skeptical VP in two minutes? / Where is the weakest link in the trust chain today?